

Test Execution Report — REST API Testing using Postman

Prepared By: Boopathi T

Date: September 20, 2026

Tool Used: Postman v12.23.2

Project Type: API Testing (Functional + Negative)

1. Objective

This project was carried out to test the REST API endpoints exposed by JSONPlaceholder (a mock REST API used for practice and testing purposes) and to verify how the API behaves under both valid and invalid conditions.

The goal was to check whether the API returns the correct HTTP status codes, correct response structure, and correct data for each operation — GET, POST, PUT, and DELETE — and to see how the API reacts when it is given bad or incomplete input.

All requests were built and executed manually in Postman, and each request was validated using `pm.test()` assertions written in JavaScript. The full collection was then run together using the Postman Collection Runner to get one final consolidated result.

2. Test Environment

Parameter	Details
Tool	Postman
Environment Name	REST API - QA Environment
Base URL Variable	{{baseUrl}} → https://jsonplaceholder.typicode.com
Collection Name	REST API Testing Project
Collection Format	Postman Collection v2.1.0
Execution Mode	Postman Collection Runner
Iterations	1
Postman Version	12.23.2

Note: The `baseUrl` variable is stored in the environment file and referenced in every request as `{{baseUrl}}/posts`, so switching environments (e.g. QA vs Staging) does not require editing individual requests.

3. Collection Structure

The collection was organized into 5 folders, grouped by HTTP method and test type, so requests are easy to locate and re-run individually or as a full suite.

Folder	Requests Inside	Purpose
01 - GET Requests	Get All Posts, Get Single Post, Invalid Post ID	Validate read operations
02 - POST Requests	Create Post, Missing Required Fields, Invalid Data Type	Validate create operation and input handling
03 - PUT Requests	Update Post, Update Post Invalid ID	Validate update operation
04 - DELETE Requests	Delete Post, Invalid Post ID	Validate delete operation
05 - Negative Tests	Invalid Endpoint, Invalid Post ID Format, Empty Request Body, Null Values	Validate error handling and edge cases

Total Folders: 5 | **Total Requests:** 14

4. Test Cases and Results

#	Method	Request Name	Endpoint	Assertions	Status
1	GET	Get All Posts	/posts	4	PASS
2	GET	Get Single Post	/posts/1	5	PASS
3	GET	Invalid Post ID	/posts/99999	3	PASS
4	POST	Create Post	/posts	6	PASS
5	POST	Missing Required Fields	/posts	3	PASS
6	POST	Invalid Data Type	/posts	3	PASS
7	PUT	Update Post	/posts/1	7	PASS
8	PUT	Update Post Invalid ID	/posts/99999	2	PASS
9	DELETE	Delete Post	/posts/1	3	PASS
10	DELETE	Invalid Post ID	/posts/99999	3	PASS
11	GET	Invalid Endpoint	/invalid-endpoint	3	PASS
12	GET	Invalid Post ID Format	/posts/99999	3	PASS
13	POST	Empty Request Body	/posts	3	PASS
14	POST	Null Values	/posts	3	PASS

Total Test Cases: 14 | **Passed:** 14 | **Failed:** 0

5. Validations Performed

Each request carried its own `pm.test()` block, and the checks generally fell into these categories:

- HTTP status code validation — confirming the response code matched what the endpoint should return for that scenario
- Content-Type validation — confirming the response header included `application/json`
- Response structure validation — confirming the response was an object or array as expected
- Required field validation — confirming fields like `id`, `title`, `body`, `userId` were present
- Response data validation — confirming returned values matched what was sent in the request
- Negative scenario validation — confirming the API responded predictably to bad or missing input

Every request also logged the method, URL, status code, and response body to the Postman console (`console.log`) during execution, which made it easier to debug failures while building the collection.

Sample assertion — Create Post

```
pm.test("Status code should be 201", function () {
  pm.response.to.have.status(201);
});

pm.test("Response should contain generated ID", function () {
  const response = pm.response.json();
  pm.expect(response).to.have.property("id");
});

pm.test("Title should match request", function () {
  const response = pm.response.json();
  pm.expect(response.title).to.eql("QA API Test");
});
```

Sample assertion — Invalid Post ID (PUT)

```
pm.test("Invalid Post ID should not return success", function () {
  pm.expect(pm.response.code).to.be.oneOf([400, 404, 500]);
});

pm.test("Server should return an error status", function () {
  pm.expect(pm.response.code).to.be.above(399);
});
```

6. Collection Run Summary

The full collection (all 5 folders, 14 requests) was executed together using the Postman Collection Runner against the QA environment.

Metric	Result
Requests Executed	14
Assertions Executed	51
Passed	51
Failed	0
Errors	0
Pass Rate	100%
Total Execution Time	7.99 seconds
Average Response Time	381 ms

All 51 assertions passed in the final run, with no failed tests and no request errors.

7. Negative Testing Observations

Negative testing was done deliberately to see how the API reacts when it's given input it shouldn't normally accept. Since JSONPlaceholder is a mock API (it doesn't persist data or run real server-side validation), a few of its behaviors don't match what a production API would typically do. These are documented below, not as failures, but as findings worth noting.

Scenario	Request Sent	Actual Response	Observation
Invalid Post ID (GET)	GET /posts/99999	404 Not Found	Correct — matches expected behavior
Invalid Endpoint	GET /invalid-endpoint	404 Not Found	Correct — unknown route handled properly
Empty Request Body	POST /posts with {}	201 Created	Accepted without enforcing required fields
Null Values	POST /posts with null fields	201 Created	Accepted null values instead of rejecting
Invalid Data Types	POST /posts with wrong types	201 Created	No data-type checks enforced
Invalid PUT ID	PUT /posts/99999	500 Internal Server Error	Not handled gracefully
Invalid DELETE ID	DELETE /posts/99999	200 OK	No existence check before delete

Takeaway

All test cases passed because the assertions were written to match the mock API's actual (documented) behavior — not because the API is strictly validating input. In a real production API, I would expect:

- A 400 Bad Request for missing required fields or wrong data types, instead of 201 Created
- A 404 Not Found for updating or deleting a resource that doesn't exist, instead of 500 or 200

This gap is a normal characteristic of JSONPlaceholder as a mock service, but it's exactly the kind of thing worth flagging if this API were being tested ahead of a real release.

8. Observations Log

ID	Observation	Severity	Category
OBS-01	No required-field validation on POST with empty body	Low	Mock API behavior
OBS-02	Null values accepted without rejection	Low	Mock API behavior
OBS-03	No data-type enforcement on POST payload	Low	Mock API behavior
OBS-04	PUT on non-existent ID returns 500 instead of 404	Medium	Error handling gap
OBS-05	DELETE on non-existent ID returns 200 instead of 404	Medium	Error handling gap

9. Conclusion

This project covered 14 API requests across GET, POST, PUT, and DELETE methods, with 51 automated assertions built and run through Postman. The full collection executed with a 100% pass rate — 51 out of 51 assertions passed, with zero failures and zero errors, in just under 8 seconds.

Beyond just confirming the "happy path" worked, the negative test scenarios turned out to be the more interesting part of this exercise — they showed where a mock API's behavior diverges from what a real production API would be expected to do, particularly around input validation and error handling for missing resources. That distinction is documented above and would be the starting point if this were ever tested against a live production-grade service instead of a mock one.

10. Project Files

File	Description
REST_API_Testing_postman_collection.json	Postman collection with all 14 requests and test scripts
REST_API_QA_Environment_postman_environment.json	Postman environment with the baseUrl variable
Test_Execution_Report.docx / .pdf / .md	This test execution report

Report prepared and executed by Boopathi T

using Postman.